Software Engineering Report

Project: University Cafeteria Order & Loyalty System

Marwan Tamer 23101477

Nour Rashad 23101495

Adel Sobhy 23101870

Mostafa Kassab 23101602

Abstract

This system digitizes cafeteria operations for students and staff. Students can register/login, browse categorized menus, build carts, pay via Cash or Wallet, earn and redeem loyalty points, and receive order-status notifications. Staff roles (Admin, Manager, Cashier, Chef) handle onboarding, menu management, checkout, and kitchen preparation. The code follows an object-oriented design with clear separation between domain models, services, and policies/strategies, enabling maintainability and extension (e.g., adding new payment methods or offers).

Introduction

Manual cafeteria processes are error-prone and slow. The proposed system integrates ordering, payments, loyalty rewards, and basic reporting into one cohesive flow. The design emphasizes SRP and strategy interfaces to keep business logic testable and easy to evolve.

Objectives

Provide a fast, user-friendly ordering and checkout experience.

Track loyalty points and allow redemption for discounts/free items.

Support role-based staff operations: admin onboarding, menu management, cashier checkout, and chef preparation.

Generate basic reports and notify students about order status.

System Requirements:

Functional Requirements

Student registration/login (PIN-based).

Browse menu by Category; add/remove items to a cart.

Calculate totals: subtotal + Tax – Discounts → grand total.

Payments: CashPayment or WalletPayment via PaymentProcessor.

LoyaltyProgram: accrue points on paid orders; redeem points for discounts or FreeCoffee offer.

Order lifecycle: NEW → PAID → PREPARING → READY → COMPLETED/CANCELED.

Staff functions

Admin registers employees (via EmployeeRegister/Factory).

Manager manages menu items (create/update/delete).

Cashier performs checkout.

Chef updates preparation/ready states.

Notifications to students on status changes.

SystemReports/ReportManager: basic sales/points summaries.

Non-Functional Requirements

Usability: Clear CLI menus, confirmations for destructive actions.

Performance: O(number of order lines) per checkout; no heavy I/O.

Security: Hide PIN input; hash & salt PINs (recommended).

Maintainability: SRP; interfaces for menu, student repo, and payments.

Testability: Services decoupled from UI; in-memory repos for fast tests.

Abbott's Technique (from nouns → classes/components)

Student → Student, managed by StudentManager (repo interface: IStudentRepository).

Menu/Menu Item/Category/Price/T→ MenuItem, Category, Price,, managed by MenuManager (IMenuManager).

Order/Cart/Line → Order, OrderLine, state via OrderStatus.

Payment → strategy via PaymentStrategy with CashPayment and WalletPayment, coordinated by PaymentProcessor → outputs PaymentBreakdown + PaymentReceipt + TransactionRecord.

Loyalty/Reward/Offer → LoyaltyProgram + offers FreeCoffee, .

Staff → Employee + roles Admin, Manager, Cashier, Chef; onboarding via EmployeeRegister & EmployeeFactory; Role enum.

Auth → Authentication.

Notifications → NotificationService.

Reports → ReportManager, SystemReports.

(Note: RedemptionRule class exists but is not referenced anywhere in code—can be removed or properly integrated.)

SOLID Principles (How they're applied)

SRP: MenuManager handles menu only; PaymentProcessor handles payments; LoyaltyProgram handles points/rewards; ReportManager composes reports.

OCP: Add a new payment by implementing PaymentStrategy (no changes to existing logic). Add a new offer by extending loyalty policies.

LSP: Code uses abstractions (IMenuManager, IStudentRepository, PaymentStrategy) so new implementations can substitute safely.

ISP: Separate interfaces for menu, students, and payments keep classes focused.

DIP: High-level services depend on interfaces (e.g., repos/strategies), not concrete classes.

System Design (High Level)

Presentation (CLI): Main, Authentication (role-based menus)

Services: OrderProcessor, PaymentProcessor, StudentManager, MenuManager, LoyaltyProgram, NotificationService, ReportManager

Domain: Order, OrderLine, MenuItem, Price, Tax, PaymentBreakdown, PaymentReceipt, TransactionRecord, Student

Staff & Infra: Employee, Admin, Manager, Cashier, Chef, EmployeeRegister, EmployeeFactory, Role, AppConfig

Policies/Enums: PaymentStrategy (+ CashPayment, WalletPayment), OrderStatus, Category, Currency, Offer/SeasonalOffer, FreeCoffee

Implementation Notes (accurate to your code)

Languages/Stack: Java console app; in-memory data structures; clear package-level separation by responsibility.

Classes present: Admin, AppConfig, Authentication, Cashier, CashPayment, Category, Chef, Employee, EmployeeFactory, EmployeeRegister, IMenuManager, IPaymentProcessor, IStudentRepository, LoyaltyProgram, MenuManager, NotificationService, Order, OrderLine, OrderProcessor, OrderStatus, PaymentBreakdown, PaymentProcessor, PaymentReceipt, PaymentStrategy, Price, RedemptionResult, RedemptionRule (unused), ReportManager, Role, Student, StudentManager, WalletPayment, Tax, TransactionRecord, etc.

Points defaults (as used in your logic): earn ≈ 10 points per 100 EGP; 10 points ≈ 1 EGP redemption value (configurable in LoyaltyProgram).

Free items: FreeCoffee offer supported through loyalty flow.

Known dead code: RedemptionRule detected as unused (safe to remove or wire into LoyaltyProgram).

Key Use Cases

Student Checkout (Wallet)

Build cart → apply redemption (optional) → PaymentProcessor debits wallet → receipt + transaction recorded → points accrued → notify student → chef prepares.

Redeem Points

LoyaltyProgram computes discount or free item (e.g., FreeCoffee) → apply to PaymentBreakdown → pay remainder via chosen method.

Manager Updates Menu

CRUD on items via MenuManager (name/price/category) with validation and ID generation.

Admin Registers Staff

EmployeeRegister + EmployeeFactory create Cashier/Chef/Manager/Admin with IDs and PINs.

Error Handling & State

Validate wallet balance before payment; no negative quantities.

Atomic checkout: either PAID with records or unchanged.

Allow only legal transitions (e.g., PAID → PREPARING → READY → COMPLETED).

Friendly messages for missing IDs or unavailable items.

Security

Use masked input for PIN entry; store PINs hashed + salted (recommended upgrade if plain stored now).

Avoid logging sensitive data; redact identifiers in logs.

Testing Strategy

Unit: PaymentProcessor (cash/wallet), LoyaltyProgram (accrual + redemption), rounding/Tax.

Integration: OrderProcessor ↔ PaymentProcessor ↔ LoyaltyProgram.

Acceptance: End-to-end CLI scripts covering happy path and edge cases.

Conclusion & Future Work

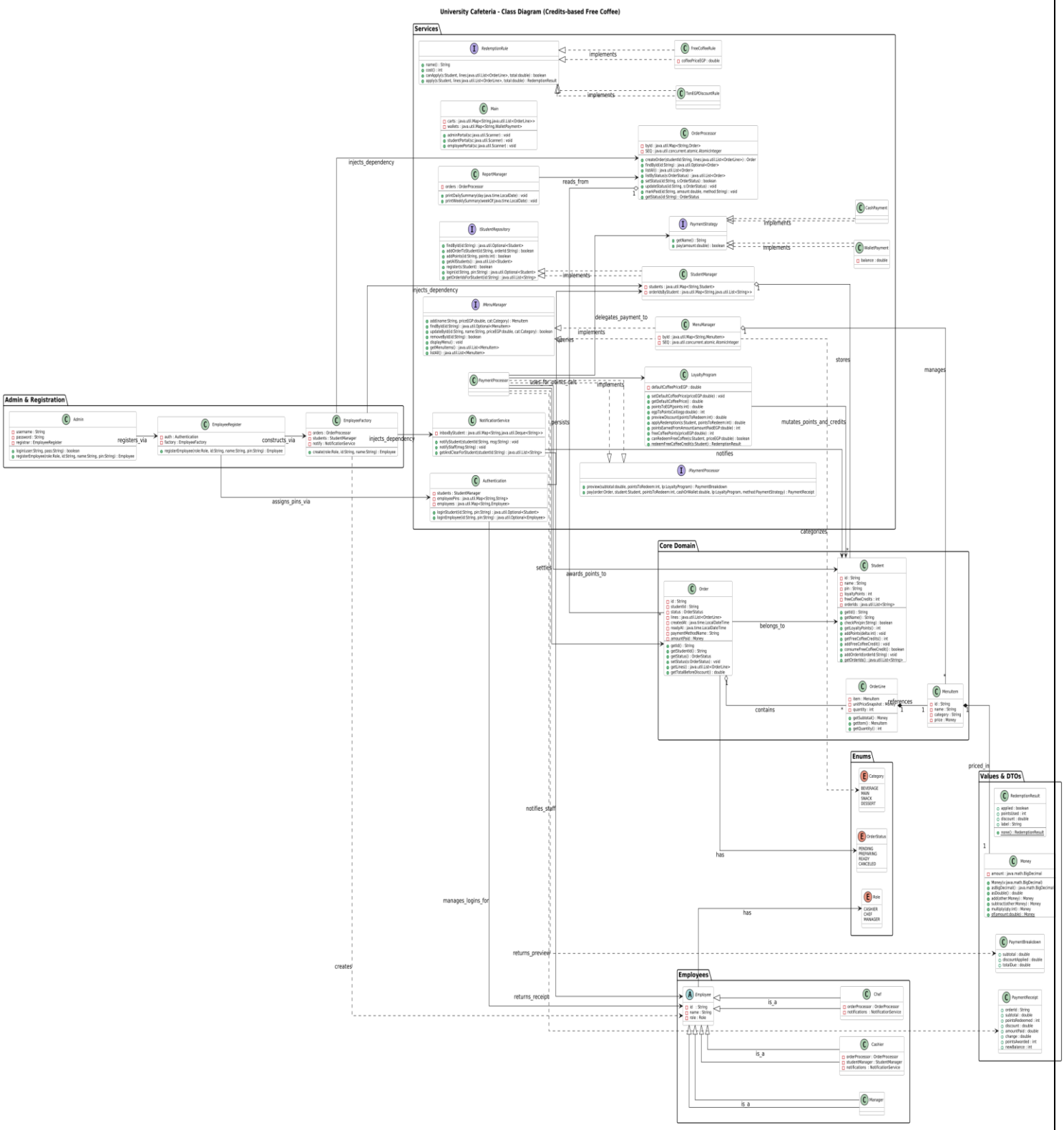
The current system cleanly implements ordering, payments, loyalty, and staff workflows with extensible strategies. Next steps:

Persist data to a lightweight DB (e.g., SQLite/H2) or JSON files.

Add REST API/GUI for a modern interface.

Expand reports (top items, hourly sales, points breakage).

Remove or integrate RedemptionRule to avoid confusion.


University Cafeteria - Class Diagram (Credits-based Free Coffee)
Services
RedemptionRule
name() : String
cost() : int
canApply(s:Student, lines:java.util.List<OrderLine>, total:double) : boolean
apply(s:Student, lines:java.util.List<OrderLine>, total:double) : RedemptionResult
implements
FreeCoffeeRule
coffeePriceEGP : double
implements
TenEGPDiscountRule
Main
carts : java.util.Map<String,java.util.List<OrderLine>>
wallets : java.util.Map<String,WalletPayment>
adminPortal(sc:java.util.Scanner) : void
studentPortal(sc:java.util.Scanner) : void
employeePortal(sc:java.util.Scanner) : void
OrderProcessor
byId : java.util.Map<String,Order>
SEQ : java.util.concurrent.atomic.AtomicInteger
createOrder(studentId:String, lines:java.util.List<OrderLine>) : Order
findById(id:String) : java.util.Optional<Order>
listAll() : java.util.List<Order>
listByStatus(s:OrderStatus) : java.util.List<Order>
setStatus(id:String, s:OrderStatus) : boolean
updateStatus(id:String, s:OrderStatus) : void
markPaid(id:String, amount:double, method:String) : void
getStatus(id:String) : OrderStatus
injects_dependency
ReportManager
orders : OrderProcessor
printDailySummary(day:java.time.LocalDate) : void
printWeeklySummary(weekOf:java.time.LocalDate) : void
reads_from
CashPayment
PaymentStrategy
getName() : String
pay(amount:double) : boolean
implements
implements
WalletPayment
balance : double
IStudentRepository
findById(id:String) : java.util.Optional<Student>
addOrderToStudent(id:String, orderId:String) : boolean
addPoints(id:String, points:int) : boolean
getAllStudents() : java.util.List<Student>
register(s:Student) : boolean
login(id:String, pin:String) : java.util.Optional<Student>
getOrderIdsForStudent(id:String) : java.util.List<String>
implements
StudentManager
students : java.util.Map<String,Student>
orderIdsByStudent : java.util.Map<String,java.util.List<String>>
injects_dependency
IMenuManager
addItem(name:String, priceEGP:double, cat:Category) : MenuItem
findById(id:String) : java.util.Optional<MenuItem>
updateById(id:String, name:String, priceEGP:double, cat:Category) : boolean
removeById(id:String) : boolean
displayMenu() : void
getMenuItems() : java.util.List<MenuItem>
listAll() : java.util.List<MenuItem>
delegates_payment_to
implements
MenuManager
byId : java.util.Map<String,MenuItem>
SEQ : java.util.concurrent.atomic.AtomicInteger
stores
manages
PaymentProcessor
uses_for_points_calc
implements
LoyaltyProgram
defaultCoffeePriceEGP : double
setDefaultCoffeePrice(priceEGP:double) : void
getDefaultCoffeePrice() : double
pointsToEGP(points:int) : double
egpToPoints(aEnkgp:double) : int
previewDiscount(points:ToRedeem:int) : double
applyRedemption(s:Student, points:ToRedeem:int) : double
pointsEarnedFromAmount(amountPaidEGP:double) : int
freeCoffeePrice(priceEGP:double) : int
canRedeemFreeCoffee(s:Student, priceEGP:double) : boolean
redeemFreeCoffee(Credits:Student) : RedemptionResult
persists
mutates_points_and_credits
Admin & Registration
Admin
username : String
password : String
register : EmployeeRegister
loginUser(username:String, pass:String) : boolean
registerEmployee(role:Role, id:String, name:String, pin:String) : Employee
registers_via
EmployeeRegister
auth : Authentication
factory : EmployeeFactory
registerEmployee(role:Role, id:String, name:String, pin:String) : Employee
constructs_via
EmployeeFactory
orders : OrderProcessor
students : StudentManager
notify : NotificationService
create(role:Role, id:String, name:String) : Employee
injects_dependency
NotificationService
inboxByStudent : java.util.Map<String,java.util.Deque<String>>
notifyStudent(studentId:String, msg:String) : void
notifyStaff(msg:String) : void
getAndClearForStudent(studentId:String) : java.util.List<String>
notifies
IPaymentProcessor
preview(subtotal:double, pointsToRedeem:int, lp:LoyaltyProgram) : PaymentBreakdown
pay(orderOrder, student:Student, pointsToRedeem:int, cashOnWallet:double, lp:LoyaltyProgram, method:PaymentStrategy) : PaymentReceipt
Authentication
students : StudentManager
employeePins : java.util.Map<String,String>
employees : java.util.Map<String,Employee>
loginStudent(id:String, pin:String) : java.util.Optional<Student>
loginEmployee(id:String, pin:String) : java.util.Optional<Employee>
assigns_pins_via
categorizes
Core Domain
settles
awards_points_to
Order
id : String
studentId : String
status : OrderStatus
lines : java.util.List<OrderLine>
createdAt : java.time.LocalDateTime
readyAt : java.time.LocalDateTime
paymentMethodName : String
amountPaid : Money
getId() : String
getStudentId() : String
getStatus() : OrderStatus
setStatus(s:OrderStatus) : void
getLines() : java.util.List<OrderLine>
getTotalBeforeDiscount() : double
belongs_to
Student
id : String
name : String
pin : String
loyaltyPoints : int
freeCoffeeCredits : int
orderIds : java.util.List<String>
getId() : String
getName() : String
checkPin(pin:String) : boolean
getLoyaltyPoints() : int
addPoints(delta:int) : void
getFreeCoffeeCredits() : int
addFreeCoffeeCredit() : void
consumeFreeCoffeeCredit() : boolean
addOrderId(orderId:String) : void
getOrderIds() : java.util.List<String>
contains
OrderLine
item : MenuItem
unitPriceSnapshot : Money
quantity : int
getSubtotal() : Money
getItem() : MenuItem
getQuantity() : int
references
MenuItem
id : String
name : String
category : String
price : Money
Enums
Category
BEVERAGE
MAIN
SNACK
DESSERT
OrderStatus
PENDING
PREPARING
READY
CANCELLED
Role
CASHIER
CHEF
MANAGER
has
has
priced_in
Values & DTOs
RedemptionResult
applied : boolean
pointsUsed : int
discount : double
label : String
none() : RedemptionResult
Money
amount : java.math.BigDecimal
Money(v:java.math.BigDecimal)
asBigDecimal() : java.math.BigDecimal
asDouble() : double
add(other:Money) : Money
subtract(other:Money) : Money
multiply(pty:int) : Money
of(amount:double) : Money
PaymentBreakdown
subtotal : double
discountApplied : double
totalDue : double
PaymentReceipt
orderId : String
subtotal : double
pointsRedeemed : int
discount : double
amountPaid : double
change : double
pointsAwarded : int
newBalance : int
notifies_staff
manages_logins_for
returns_preview
creates
returns_receipt
Employees
Employee
id : String
name : String
role : Role
is_a
Chef
orderProcessor : OrderProcessor
notifications : NotificationService
is_a
Cashier
orderProcessor : OrderProcessor
studentManager : StudentManager
notifications : NotificationService
is_a
Manager